

IncidentIQ

Hackathon Implementation Plan

Multi-agent DevOps Incident Analysis Suite using Gradio, FastAPI, LangGraph, n8n workflow automation, and RAG.

Purpose

This document converts the final architecture diagram into an implementation plan optimized for a seven-person hackathon team submitting tomorrow. The plan favors a working, hosted, judge-friendly demo over production completeness.

Final Architecture Mapping

Architecture Area	Implementation Choice	Hackathon Behavior
Input	Gradio manual upload plus n8n-triggered automated log sources.	User uploads logs; optional Datadog/S3/Email trigger calls n8n webhook.
Processing	FastAPI exposes the backend; LangGraph manages agent state and routing.	Parser/classifier, timeline, RCA, remediation, critic, notification, ticket, cookbook.
RAG Engine	Runbook/SOP retrieval enriches remediation and RCA.	Lexical/local fallback first; OpenAI embeddings + Weaviate/FAISS optional.
Integrations	n8n switch routes high severity incidents to Slack/JIRA.	Preview-first payloads; real writes only when credentials and approval are present.
Output	Traceable, actionable incident summary.	Markdown/JSON/CSV reports, Slack message, JIRA ticket payload, cookbook checklist.

Current Repository State

- The src/ directory has the intended module folders but no production implementation yet.
- The latest usable implementation is the SRC-aligned v4 notebook, which should be extracted into src/ modules.
- Existing docs already define scoring priorities: concept score, difficulty, and code quality.
- The final uploaded diagram supersedes older frontend assumptions: Gradio is the frontend and n8n is the workflow/integration orchestrator.

Shared System Contract

Every teammate should build toward the same response object so Gradio, FastAPI, n8n, agents, reporting, and RAG can work in parallel.

```
{
  "run_id": "string",
  "status": "completed|failed|needs_approval",
  "highest_severity": "P1|P2|P3|Unknown",
  "incidents": [],
```

```

    "slack_payloads": [],
    "jira_payloads": [],
    "cookbooks": [],
    "reports": {
      "markdown": "path",
      "json": "path",
      "csv": "path"
    },
    "eval": {
      "enabled": false,
      "summary": null
    }
  }
}

```

MVP Scope

- Gradio public upload dashboard.
- FastAPI /health and /analyze endpoints.
- Deterministic log parsing, signal extraction, clustering, and fallback reasoning.
- LangGraph-style multi-agent orchestration.
- RAG context from local SOP/runbook documents.
- Slack and JIRA preview payloads, optionally sent by n8n.
- n8n workflow export showing severity-based routing.
- Markdown/JSON/CSV report outputs.
- README, setup commands, screenshots, and under-5-minute demo video.

Deliberate De-scope

- Do not block submission on real Slack/JIRA credentials; preview payloads are acceptable.
- Do not block submission on Weaviate; lexical/local RAG is sufficient for MVP.
- Do not require LLM API keys for the demo; deterministic fallback must work.
- Do not build full FastAPI plus separate database persistence; file-based outputs are enough.
- Do not add large zip-only artifacts to GitHub; commit source code and small sample data references.

Implementation Phases

Phase	Timebox	Outcome
0. Freeze	0-1 hour	Lock architecture, branches, output contract, and sample dataset.
1. Core	1-4 hours	Extract v4 notebook logic into src/ and get deterministic pipeline working.
2. Agents + RAG	2-6 hours	Add LangGraph runner, agent outputs, and runbook retrieval.
3. Interfaces	3-7 hours	Wire FastAPI, Gradio, and n8n to the same response schema.
4. Hosting	6-8 hours	Deploy Gradio/FastAPI and import n8n workflow.
5. Polish	8-10 hours	README, screenshots, demo script, smoke tests, final video.
6. Freeze	Final hour	Only fix blockers; submit repo, URL, workflow export, and video.

Hosting Recommendation

- Use Hugging Face Spaces with Docker for the public Gradio demo if possible.
- Run FastAPI in the same container or expose it via Render free tier if splitting services.
- Use n8n Cloud trial/free workspace for the workflow and export the workflow JSON.
- Keep secrets optional through .env.example and feature flags.

Critical Environment Flags

```
ENABLE_LLM_MODE=false
ENABLE_RAG_MODE=true
ENABLE_PREVIEW_MODE=true
ENABLE_REAL_INTEGRATIONS=false
OPENAI_API_KEY=
OPENAI_BASE_URL=
MODEL_NAME=
SLACK_WEBHOOK_URL=
JIRA_BASE_URL=
JIRA_EMAIL=
JIRA_API_TOKEN=
```

Definition of Done

- A judge can open a public URL, upload a sample log file, and see incidents plus RCA/remediation outputs.
- The demo visibly uses multiple agents, RAG context, and n8n routing.
- The project runs without paid services or API keys using fallback mode.
- The README explains setup, architecture, concepts used, and judging value.
- The demo video is under five minutes.